

METU Computer Engineering

# CEng 111 – Fall 2026
# Week 7b

**Credit**: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.





Previously on CENG111!

# Lists in Python

```
>>> [1, 2, 3, 4, "a"]
[1, 2, 3, 4, 'a']
>>> type([1, 2, 3, 4, "a"])
<type 'list'>
```

- Lists in Python: collection of data enclosed in brackets, separated by comma.
- Accessing elements of a list (like strings & tuples):

- Positive Indexing: `[1, 2, 3, 4, "a"][2]` returns 3.
- Negative Indexing: `[1, 2, 3, 4, "a"][-1]` returns `'a'`.
- Ranged Indexing, *i.e.*, `[start:end:step]`: `[1, 2, 3, 4, "a"][0:4:2]` leads to `[1, 3]`.

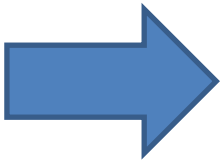


Previously on CENG111!

# Variables in Python

METU Computer Engineering

```
>>> a = 4
>>> b = 3
>>> c = a + b
>>> a
4
>>> b
3
>>> c
7
```



- We don’t need to declare a variable before using it.
- We don’t need to specify the type of a variable.

- ‘=‘ means “Change the content of the variable with the value at the right-hand side”.
  - Assignment!
- The left-side of the assignment should be a valid variable name:
  - Ex: a+2 = 5 → NOT VALID!



METU Computer Engineering

Previously on CENG111! v2

# Variable Naming in Python

- Variable names are case sensitive. So, the names `a` and `A` are two different variables.
- Variable names can contain letters from the English alphabet, numbers and an underscore `_`.
- Variable names can only start with a letter or an underscore. So, `10a`, `$a`, and `var$` are all invalid whereas `_a` and `a_20`, for example, are valid names in Python.



# Today



- Aliasing



# Administrative Notes



- Midterm Exam
  - Date: 20 November, 18:00
- Lab Exams
  - Lab Exam 1: Held on 8 November, Sunday, 13:00
  - Lab Exam 2: 6 December, Sunday, 13:00
  - Lab Exam 3: 20 December, Sunday, 13:00
  - Lab Exam 4: 3 January, Sunday, 13:00



# Variables, Values and Aliasing in Python

METU Computer Engineering

- Every data (whether constant or not) has an identifier (an integer) in Python:

```
>>> a = 1
>>> b = 1
>>> id(1)
135720760
>>> id(a)
135720760
>>> id(b)
135720760
```

- If the type of the data is mutable, there is a problem!!!

```
>>> a = ['a', 'b']
>>> b = a
>>> id(a)
3083374316L
>>> id(b)
3083374316L
>>> b[0] = 0
>>> a
[0, 'b']
```

**This is called Aliasing.**

```
a = 4
b = [1,2,3,a]
a = 8
print b
```

```
>>> a=[1,2]
>>> b=[1,2,a]
>>> a
[1, 2]
>>>
>>> b
[1, 2, [1, 2]]
>>> a.append(3)
>>> b
[1, 2, [1, 2, 3]]
>>> a
[1, 2, 3]
```

- Check id(a) and id(b[3])



# How to make copy of the list?

METU Computer Engineering

- list(List-to-be-copied)
- L[:]
- Shallow copy

  import copy

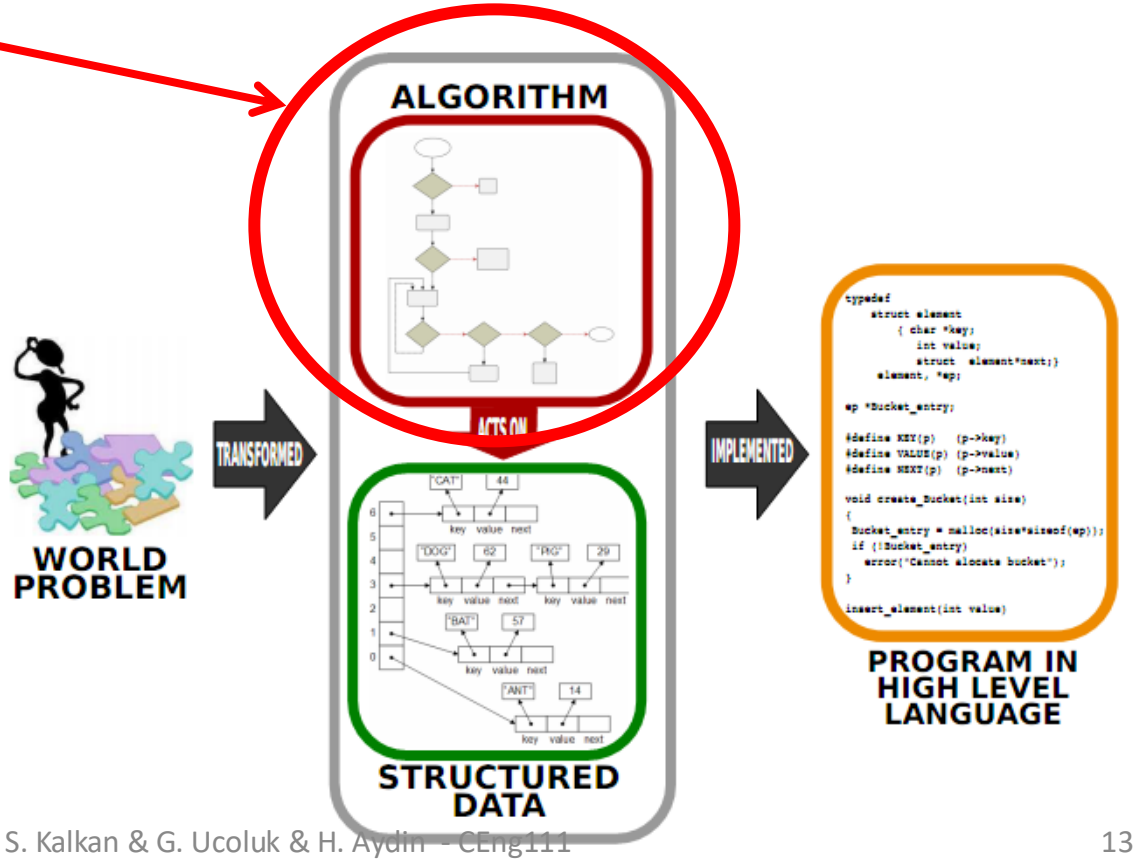
  copy.copy(list)
- Deep copy

  import copy

  copy.deepcopy(list)

# Now

- We start another ingredient of a program:
  - Actions!






METU Computer Engineering





# What are actions?

- Actions in a PL are the *things* that we can do with the data.
  - Create data or modify data
  - Interact with the external environment



# Actions for creating/modifying data



- Evaluating a mathematical expression
  - But there are differences to the expressions in Mathematics
- Working with structured data
- Storing results of computations (in another data)
- Making a decision about how to proceed with the computation
  - `if x*y < 3.1415 then` ⟨*do some action*⟩
  - `if "ali" in class_111_list then` ⟨*do some action*⟩
  - `if tall("ali") then` ⟨*do some action*⟩





# Interaction-type actions

- "Interaction" means Input/Output.
- Why interact with the environment? Why do we have Input/Output actions?
  - To react on a change in the external environment
  - To produce an effect in the external environment



# Action Types in High-Level Languages

METU Computer Engineering

- Expression evaluation
  - 3 + 4 * 5 / 2

vs.

- Statement execution
  - del L[2:4]





# EXPRESSIONS

# Expressions

- An expression is a calculation which has a set of *operations*.
- Operations have *operators* and *operands*.
- Example: 3 + 4
  - + → operator
  - 3, 4 → operands

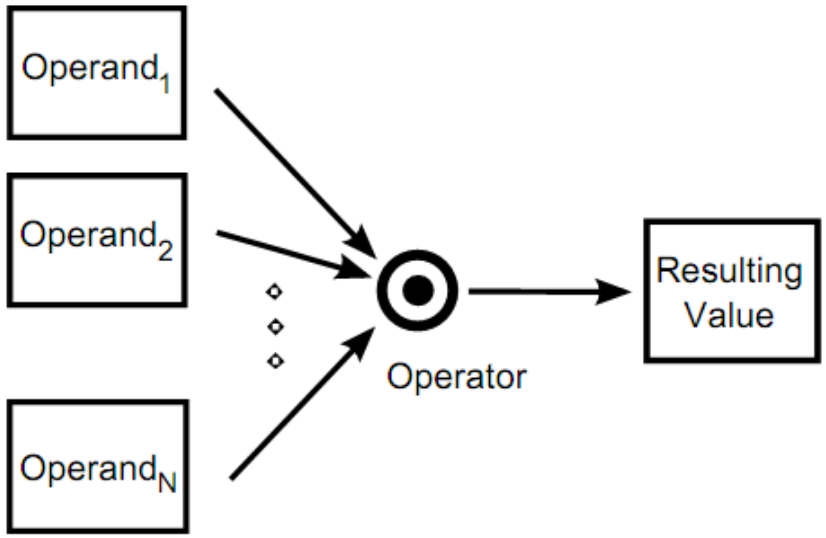
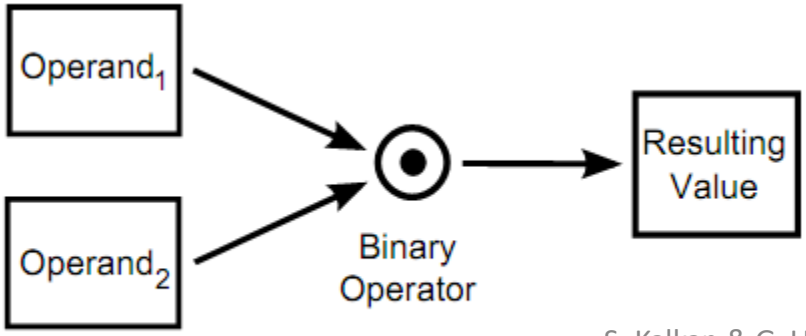
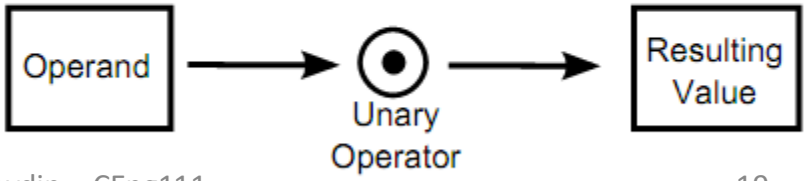



Figure 3.1: N-ary operation




(a) Binary operation




(b) Unary operation

METU Computer Engineering





# Expressions in Python
## - Involving arithmetic operators -

| Operator | Operator Type | Description |
|---|---|---|
| + | Binary | Addition of two operands |
| - | Binary | Subtraction of two operands |
| - | Unary | Negated value of the operand |
| + | Unary | Positive value of the operand |
| * | Binary | Multiplication of two operands |
| / | Binary | Division of two operands |
| ** | Binary | Exponentiation of two operands (Ex: `x**y` $= x^y$) |





# Expressions in Python
# - Involving arithmetic operators -

- Precedence & Associativity of arithmetic operators.
- What is precedence?
  - The expression “3 + 4 * 5” can have two different interpretations:
    - (3+4)*5
    - 3 + (4*5)
- What is associativity?
  - The expression “3.02 + 4.1 + 5.24” has two different interpretations:
    - (3.02+4.1)+5.24
    - 3.02+(4.1+5.24)



# Expressions in Python
# - Involving arithmetic operators -

METU Computer Engineering

- Precedence & Associativity of arithmetic operators.
- Top: highest precedence.
- Bottom: lowest precedence.

| Operator | Type | Associativity | Description |
|---|---|---|---|
| ** | Binary | Right-to-left | Exponentiation |
| +, - | Unary | Right-to-left | Positive, negative |
| *, /, //, % | Binary | Left-to-right | Multiplication, Division, Remainder, Modulo |
| +, - | Binary | Left-to-right | Addition, Subtraction |

Floor division
(fraction part of the result is removed)





# Expressions in Python
# - Involving container operators -

- Concatenation (+)
  - “a” + “b” → “ab”
- Repetition (*)
  - “a” * 3 → “aaa”
- Membership (in, not in):
  - “a” in “Mathematics” → True
  - “a” not in “Mathematics” → False
- Indexing ([])

# Expressions in Python
# - Involving container operators -



- Precedence and associativity of container operators

| Operator | Type | Associativity | Description |
|---|---|---|---|
| [] | Binary | Left-to-right | **Indexing** |
| ** | Binary | Right-to-left | Exponentiation |
| +, - | Unary | Right-to-left | Positive, negative |
| *, /, //, % | Binary | Left-to-right | Multiplication & **Repetition**, Division, Remainder, Modulo |
| +, - | Binary | Left-to-right | Addition, Subtraction, **Concatenation** |
| in, not in | Binary | Right-to-left | Membership |



# Expressions in Python
# - Involving relational operators -

METU Computer Engineering

- Equality (==)
  - Two data are equivalent if they represent the same value/information!
  - "Ali" == "Ali" → True
- Less-than (<):
  - A numerical data is less than another if the value of the first is less than that of the second:
    - 3 < 4.5 → True
  - A string is less than another if it is lexicographically (i.e., in ASCII value) less than the second.
    - "abc" < "def" → True
  - A tuple/list is less than another tuple/list if the first different items satisfy the less-than relation.

# Expressions in Python
# - Involving relational operators -

- Less-than-or-equal (<=)
  - <= → (<) or (==)
- Greater-than (>)
  - > → not (<=)
- Greater-than-or-equal-to (>=)
  - >= → not (<)
- Not-equal (!=)
  - != → not (==)



Note that in Python, relational operators can be chained. In other words, `a RO b RO c` (where `RO` is a relational operator) is interpreted as:

`(a RO b) and (b RO c)`.

In most other programming languages, `a RO b RO c` is interpreted as `(a RO b) RO c`.



METU Computer Engineering



# Expressions in Python
# - Involving relational operators -



- Precedence & Associativity

| Operator | Type | Associativity | Description |
|---|---|---|---|
| [] | Binary | Left-to-right | **Indexing** |
| ** | Binary | Right-to-left | Exponentiation |
| +, - | Unary | Right-to-left | Positive, negative |
| *, /, //, % | Binary | Left-to-right | Multiplication & Repetition, Division, Remainder, Modulo |
| +, - | Binary | Left-to-right | Addition, Subtraction, Concatenation |
| in, not in, {<, <=, >, >=, ==, !=}* | Binary | Right-to-left | Membership, **Comparison** * |

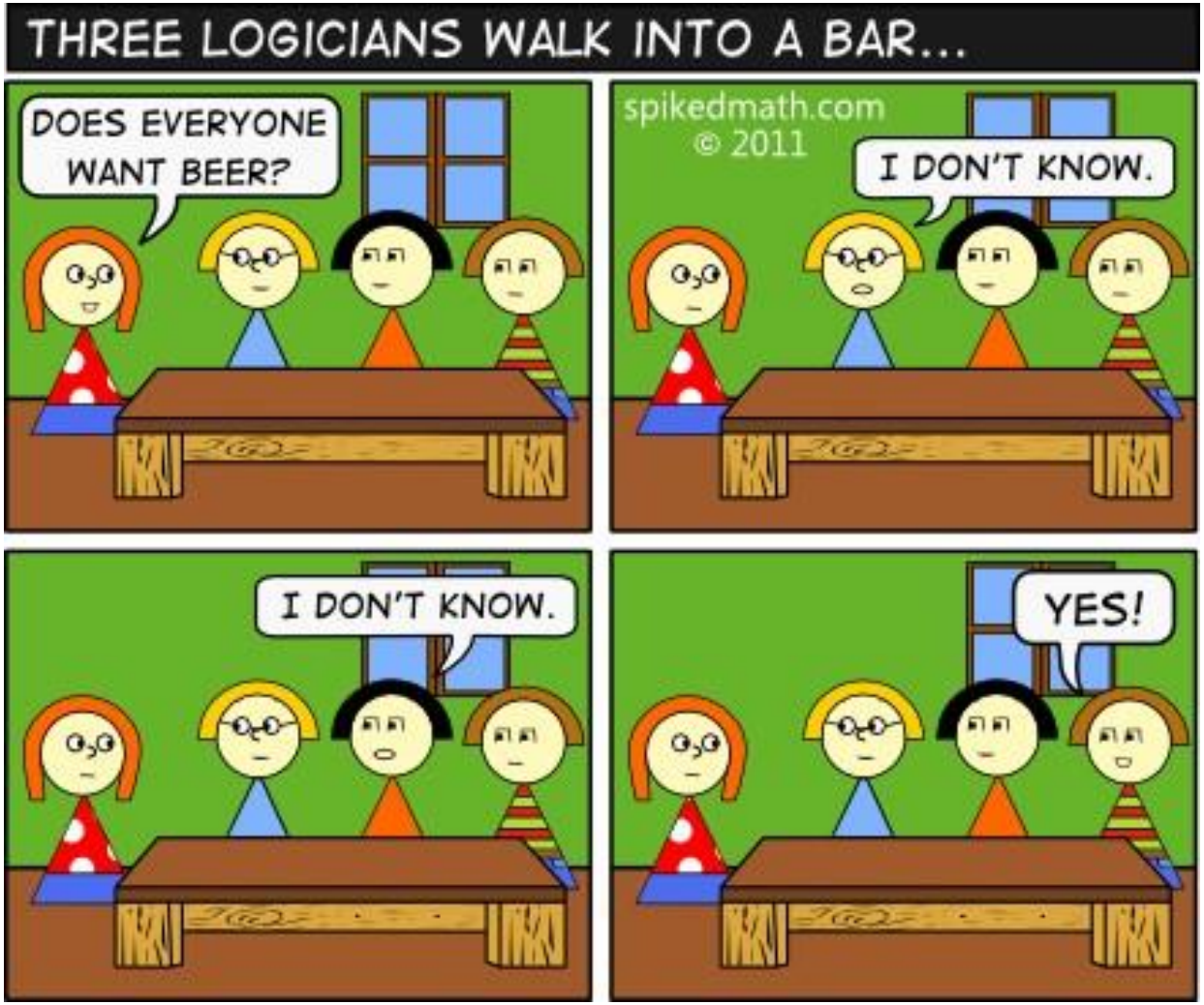
* <, <=, >, >=, ==, != have left-to-right associativity in v3





# Expressions in Python
# - Involving logical operators -

- Logical operators manipulate truth values:
- and operator
  - A and B → True iff (A is True) & (B is True)
- or operator
  - A or B → True iff either (A is True) or (B is True)
- not operator
  - not A → True iff A is False


THREE LOGICIANS WALK INTO A BAR...
DOES EVERYONE WANT BEER?
spikedmath.com
© 2011
I DON'T KNOW.
I DON'T KNOW.
YES!

# Expressions in Python
# - Involving logical operators -

- Precedence & Associativity

| Operator | Type | Associativity | Description |
|---|---|---|---|
| [] | Binary | Left-to-right | **Indexing** |
| ** | Binary | Right-to-left | Exponentiation |
| +, - | Unary | Right-to-left | Positive, negative |
| *, /, //, % | Binary | Left-to-right | Multiplication & Repetition, Division, Remainder, Modulo |
| +, - | Binary | Left-to-right | Addition, Subtraction, Concatenation |
| in, not in,{<, <= >, >=, ==, !=}* | Binary | Right-to-left | Membership, Comparison* |
| not | Unary | Right-to-left | Logical negation |
| and | Binary | Left-to-right | Logical AND |
| or | Binary | Left-to-right | Logical OR |

* <, <=, >, >=, ==, != have left-to-right associativity in v3





# Expressions in Python
## - assignment (**not** an operator) -

- Single assignment:
  - a = 4
- Multiple assignment:
  - a = b = c = 4
- Combined assignment:
  - a = a + 4 → a += 4
  - +=, *=, -=, /=, etc.

```
>>> b += 4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'b' is not defined
>>> b = 5
>>> b **= 2
>>> b
25
```

# Expressions in Python
## - walrus operator (:=) -



METU Computer Engineering

- Assignment that reflects the value:

Example:

```
>>> (a:=4) + 5
9
>>> a
4
```

- Can be used in any expression:
  - Including condition, list comprehension etc.





# List comprehension

[<Exp1> for <item> in <container>]

Example:

```
>>> [x for x in range(1, 7)]
[1, 2, 3, 4, 5, 6]
```

[<Exp1> for <item> in <container> if <condition>]

Example:

```
>>> [x for x in range(1, 7) if x % 3 == 0]
[3, 6]
```

[<Conditional-Exp1> for <item> in <container>]

Example:

```
>>> [x if x % 3 else 2*x for x in range(1, 7)]
[1, 2, 6, 4, 5, 12]
```